



START HERE

Overview

Getting started

Demo credentials

Commands

ARCHITECTURE

Services

Auth & roles

Checkout saga

Frontend

OPERATE

Infrastructure

Tools & stack

Environment & secrets

Monitoring

Testing & CI

REFERENCE

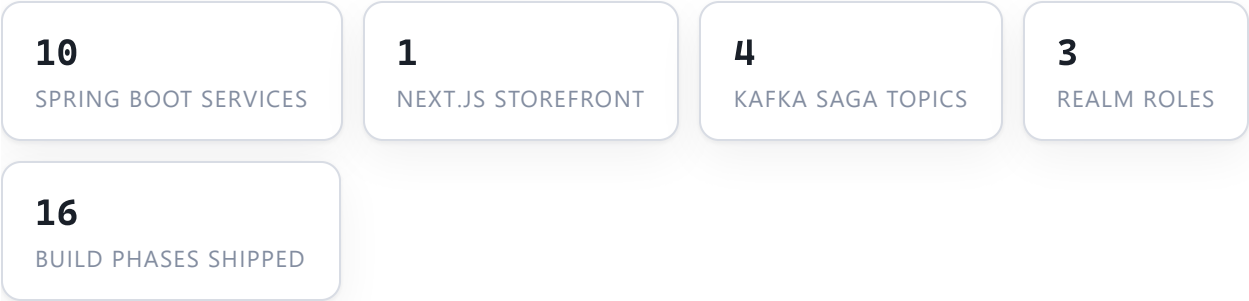
Gotchas

Build history

Locked decisions

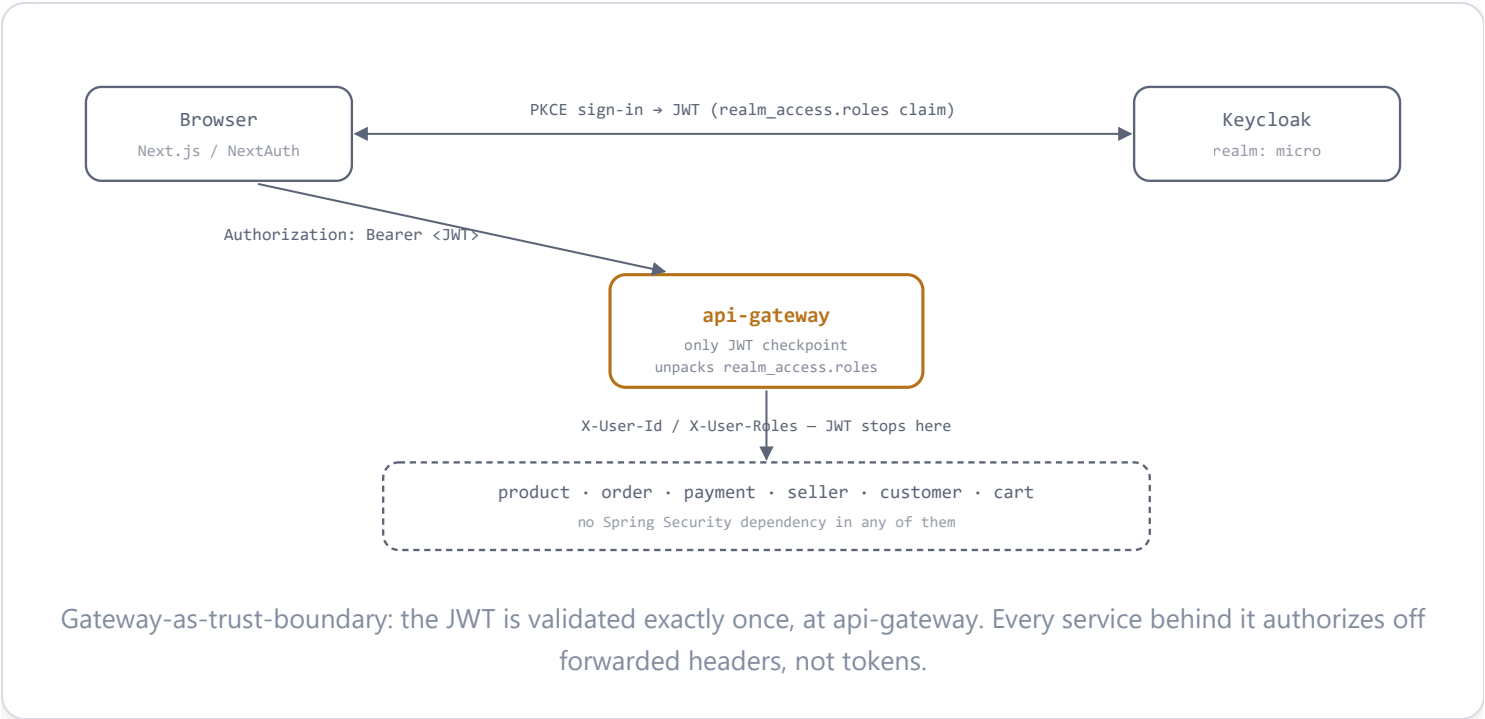
Cloth Shop Platform

A multi-vendor clothing marketplace: ten independently-deployable Spring Boot services behind one gateway, a Kafka-choreographed checkout saga, Keycloak-issued identity, and a Next.js storefront. This is the architecture as it actually stands today — what each piece does, how they talk to each other, and every command you need to build, run, test, and watch it.



01 Overview

Three audiences shop this platform: **customers** browse and buy, **sellers** list products and get paid out, **admins** moderate the marketplace and approve sellers. All three are Keycloak identities distinguished only by a realm role — there's no separate seller or admin identity system.



Three roles, one identity system

ROLE	GRANTED	CAN DO
customer	Default on sign-up	Browse, buy, review purchased products, request refunds, register as a seller
seller	Admin approves a PENDING seller registration	Everything a customer can, plus: list/edit/delete own products, fulfill own order lines, see own payouts
admin	Provisioned manually in Keycloak	Manage categories, approve/suspend sellers, view every order, everything else

02 Services

Ten independent Maven projects under `services/` — no aggregator POM, each built and run separately. Every service follows the same package layout:

`controller`, `service` + `service/serviceImpl`, `repository`, `entity` or `document`, `dto`, `mapper`, `handler` (`GlobalExceptionHandler`). The exception package is spelled `exception` everywhere except `customer-service` and `product-service`, which kept an original typo (`exeption`) — match whichever a given service already uses.

discovery-service

:8761

Eureka server

Every other service registers here on boot. Nothing calls it directly except for service discovery.

config-server

:8888

Spring Cloud Config

Centralized `application.yml` per profile, backed by an external git repo (`e-com-app-config`) — not part of this working directory. Every other service does `optional:configserver:http://localhost:8888` on boot; being `optional`, a service still starts (on its own local defaults) if config-server is unreachable.

api-gateway

:8222

Spring Cloud Gateway

Entry point

OAuth2 resource server validating JWTs against Keycloak. The **only** place a JWT gets validated — see Auth & roles. `UserContextGatewayFilter` forwards `X-User-Id` / `X-User-Roles` downstream, stripping any inbound values first so a client can't spoof them by calling a service directly. Also carries CORS, a Redis-backed rate limiter (20 req/s, burst 40), a circuit breaker + retry (GET only) on the default filter chain, and its own `/actuator/prometheus` for scraping.

Route table itself lives entirely in the external `e-com-app-config` repo — nothing in this working directory defines a single gateway route.

customer-service

:8091

MongoDB

Customer + address documents. Called synchronously by order-service

(**CustomerClient** , Feign) to validate a customer exists before an order is created — the one required cross-service call the checkout saga still makes before going async.

product-service

:8050

Postgres

Feign: order-service

The catalog. A **Product** (name / description / price / category / **sellerId**) has many **ProductVariant**s (sku / size / color / **availableQuantity**, optimistic-locked via **@Version**) — variants are the purchasable, stock-tracked unit, not the product itself.

GET /api/v1/products public; **?sellerId=** for "my catalog"

POST /api/v1/products seller/admin, ownership-checked

PUT /api/v1/products/{id} owner or admin only

DELETE /api/v1/products/{id} owner or admin only

GET /api/v1/categories public reads, admin-only writes; non-empty category can't be deleted

POST /api/v1/products/{id}/images/upload-url presigned
MinIO PUT
URL; bytes
never touch
this service

POST /api/v1/products/{id}/images registers the uploaded
object as a
ProductImage row

POST /api/v1/products/{id}/reviews gated on verified
purchase, not a role

GET /api/v1/products/{id}/reviews public

Reviews are gated by a Feign call to order-service's internal **purchased-variants** endpoint, cross-referenced against the product's own variant ids — **fails closed** if order-service is unreachable, so an outage can never manufacture fake social proof. **averageRating** / **reviewCount** are **null** / **0** (no badge shown) for unreviewed products — never fabricated.

seller-service

:8095

Postgres

Keycloak Admin API

Stripe Connect

Seller (`keycloakUserId` , business profile, `SellerStatus`
PENDING/ACTIVE/SUSPENDED, Stripe Connect fields).

POST	/api/v1/sellers/register	any authenticated user
PATCH	/api/v1/sellers/{id}/status	admin; ACTIVE grants the Keycloak <code>seller</code> role, SUSPENDED revokes it
POST	/api/v1/sellers/me/stripe/onboarding-link	creates a Stripe Express account on first call
POST	/api/v1/sellers/webhooks/stripe	signature-verified, not JWT-authenticated — Stripe calls it directly
GET	/api/v1/sellers/lookup/{keycloak-user-id}	internal only, Feign — payment-service calls this during payout settlement

order-service

:8070

Postgres

Saga orchestrator

Order + order-line orchestration. `POST /api/v1/orders` validates the customer synchronously (Feign), persists the order as `PENDING_PAYMENT`, then hands off to the async saga — see Checkout saga.

POST `/api/v1/orders` honors `Idempotency-Key` header

GET `/api/v1/orders/mine` customer-scoped, off `X-User-Id`

GET `/api/v1/orders` unfiltered — `ROLE_ADMIN` only

POST `/api/v1/orders/{id}/refund` owner or admin; the one synchronous order→payment call

PATCH `/api/v1/order-lines/{id}/fulfillment` seller/admin; moves the line to SHIPPED/DELIVERED, `Order.status` derives from all lines

GET `/api/v1/order-lines/mine` a seller's own lines across every order

GET `/api/v1/order-lines/purchased-variants/{customer-id}` internal only — product-service's review gate calls this

GET `/api/v1/coupons/preview` any authenticated user; admin CRUD on the rest of `/coupons`

payment-service

:8060

Postgres

Stripe

Charges via Stripe (`StripePaymentService`), triggered off the saga. `stripe.secret-key` reads `STRIPE_SECRET_KEY` — empty by default, so charges fail closed with a clear reason until a real test key is configured. On a successful charge, also writes one `SellerPayout` ledger row per seller in the order (gross / commission / net, default 10% commission).

GET `/api/v1/payouts` sellers see only their own; admins see all

POST `/api/v1/payouts/settle` admin, or a daily `@Scheduled` job
— moves money via Stripe
Transfer

POST `/api/v1/payments/refund` called by order-service, not the
browser

notification-service

:8040

MongoDB

Consumes Kafka events to send emails (Spring Mail + Thymeleaf, tested locally against MailDev). Payment emails are guarded on `success` — a failed payment is logged, never emailed.

cart-service

unconfirmed

Redis

A `cart:{userId}` Redis hash keyed by variant id — no JPA/Mongo at all. **POST** `/api/v1/cart/{userId}/checkout` calls order-service via Feign and clears the cart on success.

03 Auth & roles

Keycloak realm `micro`, three realm roles (`customer` default, `seller`, `admin`), provisioned once via `keycloak/setup-realm-roles.sh` (idempotent).

Becoming a seller

- 1. `POST /api/v1/sellers/register` (any signed-in user) → seller-service creates a `PENDING Seller` row.
- 2. An admin calls `PATCH /api/v1/sellers/{id}/status` with `ACTIVE` .
- 3. seller-service calls Keycloak's Admin API to grant the `seller` realm role.
- 4. The user's **next** token — not their current one, roles are baked in at token-issue time — carries `ROLE_SELLER` .

WHY HEADERS, NOT RE-VALIDATED TOKENS

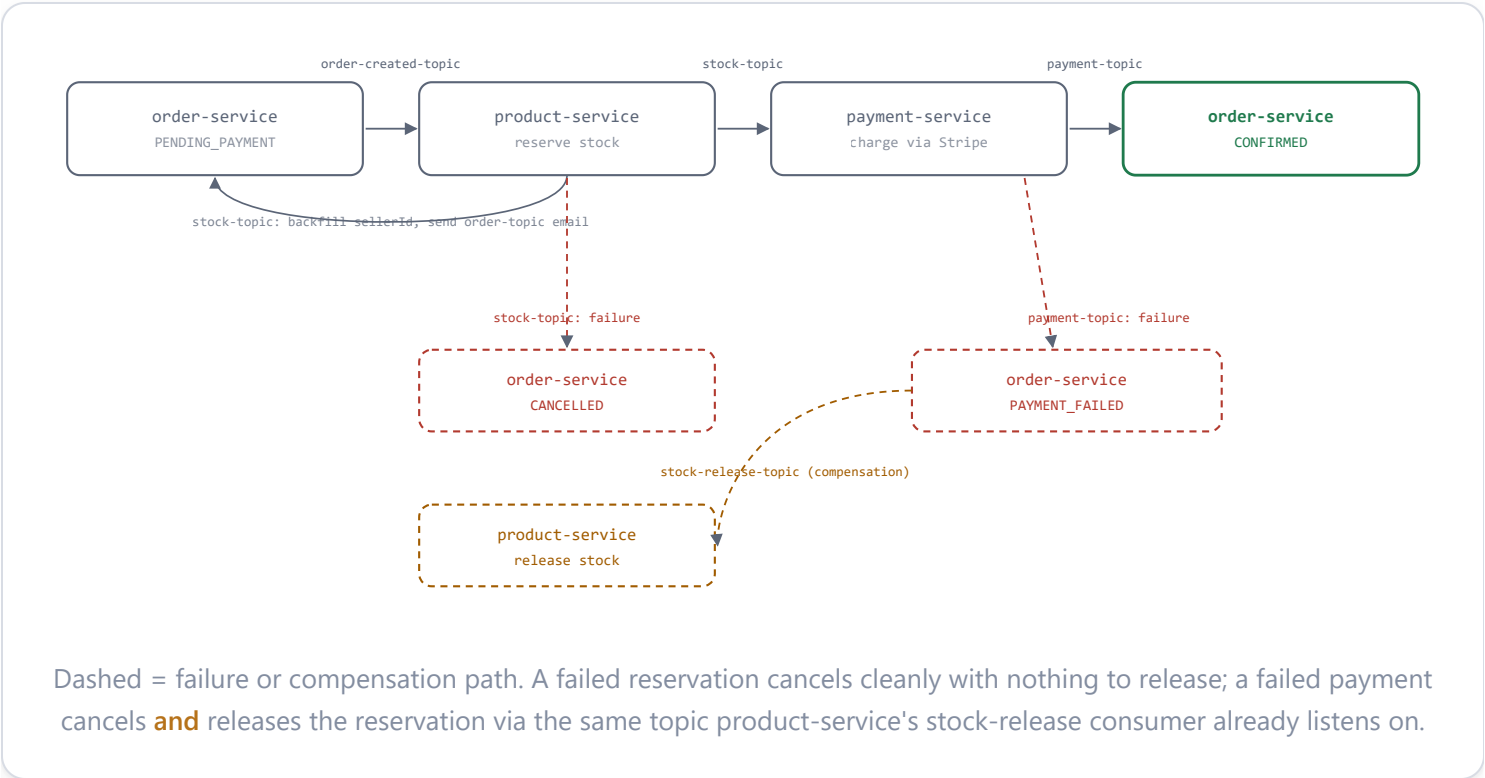
Downstream services do not validate JWTs themselves — there's no Spring Security dependency in product-service, order-service, seller-service, etc. at all. This avoids duplicating OAuth2 resource-server config into nine more services; the tradeoff is that every one of them trusts the gateway completely, which is exactly why the gateway strips inbound `X-User-Id` / `X-User-Roles` before setting its own.

PROVISIONING SCRIPTS (`KEYCLOAK/`)

SCRIPT	PROVISIONS
<code>setup-realm-roles.sh</code>	The three realm roles
<code>setup-nextjs-client.sh</code>	Public PKCE client <code>nextjs-storefront</code> (auth code + PKCE, no secret)
<code>setup-seller-service-client.sh</code>	Confidential client seller-service authenticates to Keycloak's Admin API as — needs <code>manage-users</code> , <code>view-users</code> , and <code>view-realm</code> on <code>realm-management</code>

04 Checkout saga

`order-service` → `product-service` → `payment-service` → back to `order-service`, choreographed over Kafka and correlated by `orderReference` (a string — deliberately **not** the numeric order id, which is technically client-suppliable on the request and so can't be trusted as a correlation key).



TOPIC	PUBLISHER	CONSUMER(S)	PAYLOAD
order-created-topic	order-service	product-service	<code>OrderCreatedEventDTO</code> — customer, requested variants, amount, payment method, optional <code>stripePaymentMethodId</code>
stock-topic	product-service	order-service, payment-service	<code>StockReservationResultEventDTO</code> — success/failure, resolved product/variant details, each line's <code>sellerId</code>
payment-topic	payment-service	order-service, notification-service	<code>success</code> / <code>reason</code> fields
stock-release-topic	order-service	product-service	<code>StockReleaseEventDTO</code> — reused for both payment-failure compensation and refund compensation
order-topic	order-service	notification-service	order-confirmation email trigger

REAL LIMITATION, NOT FIXED YET

No consumer-side dedup for redelivered Kafka messages — at-least-once delivery could in theory double-reserve or double-charge on redelivery. Each service keeps its own local copy of the shared event DTOs (no shared library between services); field names must match what the producer actually emits, since Jackson silently drops unmatched fields rather than erroring.

Idempotency for the initial HTTP call: `POST /api/v1/orders` honors an `Idempotency-Key` header, unique-constrained on `Order.idempotencyKey` — a repeated key returns the existing order id without repeating the customer check or re-publishing the created event.

05 Frontend

`frontend/` is a separate Next.js 16 app (App Router, Turbopack, TypeScript, Tailwind) — not a Maven service. It's the **only** client of `api-gateway`; nothing else in the repo calls the gateway from outside.

ROUTE MAP

ROUTE	ACCESS	PURPOSE
<code>/</code>	Public	Catalog, search, category filter
<code>/products/[id]</code>	Public	PDP — variants, reviews, JSON-LD structured data
<code>/wishlist</code>	Public (client-only)	localStorage, no backend
<code>/cart</code> , <code>/checkout/[orderId]</code>	Signed-in	Cart + post-checkout order status
<code>/orders</code>	Signed-in	Customer's own orders + refund action
<code>/seller/*</code>	Signed-in, gated by <code>SellerStatusGate</code>	overview, products, orders (fulfillment), payouts, onboarding
<code>/admin/*</code>	Signed-in, gated by <code>AdminGate</code>	categories, sellers (moderation), orders (read-only)

KEY MODULES

- `src/auth.ts` — NextAuth v5 with a Keycloak PKCE public client. Decodes `realm_access.roles` out of the access token into `session.roles` (no verification needed — the token was already validated by Keycloak issuing it, and every backend call re-validates it anyway).
- `src/proxy.ts` — Next 16 renamed `middleware.ts`; gates `/cart`, `/checkout`, `/account`, `/orders`, `/seller`, `/admin`.
- `src/lib/api.ts` — the only place that calls the gateway. Catalog reads are public; everything else needs the session's access token.
- `src/lib/actions.ts` — Server Actions for every mutation (cart, checkout, reviews, fulfillment, admin writes) — no client-side fetch for mutations.
- `src/lib/seo.ts`, `sitemap.ts`, `robots.ts` — site metadata, a catalog-driven sitemap that degrades to just the homepage on fetch failure, and a robots file that disallows everything auth-gated.

IF YOU TOUCH `SRC/AUTH.TS` OR THE `NEXT-AUTH .D.TS`

TypeScript module augmentation for `next-auth` / `next-auth/jwt` is fragile — the augmenting `.d.ts` needs its own `export {}` or it becomes an ambient script that *replaces* the real module's types instead of

merging with them. Both `next-auth/jwt` and `@auth/core/jwt` need the same fields augmented, since `@auth/core` 's callback signatures import `JWT` from the latter internally.

THIS IS NOT THE NEXT.JS YOU KNOW

Next 16 has real breaking changes from most training data: renamed `proxy.ts` , `fetch` no longer cached by default, `params` / `searchParams` are Promises. Check `frontend/node_modules/next/dist/docs/` before assuming an API.

06 Infrastructure & ports

`docker-compose.yml` at the repo root brings up everything the services depend on. The 10 Spring Boot services themselves are **not** containerized — they run on the host via `mvn spring-boot:run` / `java -jar`. Only infra lives in Docker.

COMPONENT	PORT	NOTES
Postgres	5432	user/pass from <code>POSTGRES_USER</code> / <code>POSTGRES_PASSWORD</code>
pgAdmin	8087	
MongoDB	27017	user/pass from <code>MONGO_ROOT_USER</code> / <code>MONGO_ROOT_PASSWORD</code>
mongo-express	8081	
Kafka	9092	+ Zookeeper 22181
Redis	6379	cart-service + gateway rate limiter
MailDev	10081 / 1025	UI / SMTP
Zipkin	9411	distributed tracing
Keycloak	8080	realm <code>micro</code> , <code>start-dev</code> mode
MinIO	9000 / 9001	API / console — <code>product-images</code> bucket auto-created, public-read
Prometheus	9090	scrapes host-run services via <code>host.docker.internal</code>
Grafana	4000	moved off 3001 — see Gotchas
kafka-exporter	9308	Kafka consumer-lag metrics for Prometheus
Frontend (dev)	3000	<code>npm run dev</code>

SERVICE PORTS (HOST-RUN)

SERVICE	PORT
discovery-service	8761
config-server	8888
api-gateway	8222
customer-service	8091
product-service	8050
seller-service	8095
order-service	8070
payment-service	8060
notification-service	8040
cart-service	unconfirmed – see Gotchas

Startup order matters: config-server → discovery-service, then the rest.

BRING UP ALL INFRA

```
docker compose up -d
```


07 Tools & stack

LAYER	TOOLS
Backend runtime	Java 21, Spring Boot 3.5.13, Spring Cloud 2025.0.2
Service mesh	Eureka (discovery), Spring Cloud Config, Spring Cloud Gateway, OpenFeign, Resilience4j
Messaging	Apache Kafka (choreographed saga)
Data stores	Postgres (JPA/Hibernate), MongoDB (Spring Data Mongo), Redis (cart, rate limiting)
Identity	Keycloak (OIDC/OAuth2, PKCE), realm roles
Payments	Stripe (charges + Stripe Connect Express for seller payouts)
Object storage	MinIO (S3-compatible, self-hosted) for product images
Observability	Micrometer + Prometheus, Grafana, kafka-exporter, Zipkin tracing
Frontend	Next.js 16 (App Router, Turbopack), TypeScript, Tailwind CSS, NextAuth v5
Testing	JUnit 5 + Mockito (unit), Testcontainers (Postgres + Kafka integration), Playwright (e2e)
CI	Jenkins (declarative <code>Verify</code> stage + shared-lib <code>buildService</code>)
Email (dev)	MailDev (SMTP catcher) + Thymeleaf templates

08 Commands

Backend – per service, from inside `services/<name>/`

There is no root aggregator POM — every command below runs from inside one service's directory.

```
mvn clean install          # build + test
mvn test                   # run all tests for this service
mvn test -Dtest=ClassName#methodName # run a single test
mvn verify                 # test + Testcontainers integration tests (what CI
mvn spring-boot:run        # run the service locally
```

ORDER-SERVICE ONLY

`saga/OrderSagaIntegrationTest.java` spins up real Postgres + Kafka via Testcontainers — needs Docker running locally. Run it alone with `mvn test -Dtest=OrderSagaIntegrationTest`.

Frontend – from inside `frontend/`

```
npm run dev                # local dev server, :3000
npm run build               # production build
npm run lint               # ESLint
npx tsc --noEmit           # type-check only
npm run test:e2e           # Playwright – needs `npx playwright install chromium` once first
```

Copy `.env.local.example` to `.env.local` before first run. `e2e/customer-checkout.spec.ts` / `e2e/seller-flow.spec.ts` additionally need `E2E_CUSTOMER_USERNAME` / `PASSWORD` and `E2E_SELLER_USERNAME` / `PASSWORD` for seeded Keycloak test accounts — they `skip` without them.

Infrastructure

```
docker compose up -d          # bring up everything
docker compose up -d --force-recreate zookeeper kafka # fixes stale-ephemeral-node Kafka/Z
docker compose config -q      # validate the compose file (no daemon nee
```

Keycloak one-time setup

```
docker compose up -d keycloak
cd keycloak
KEYCLOAK_ADMIN_USER=admin KEYCLOAK_ADMIN_PASSWORD=admin ./setup-realm-roles.sh
KEYCLOAK_ADMIN_USER=admin KEYCLOAK_ADMIN_PASSWORD=admin ./setup-nextjs-client.sh
KEYCLOAK_ADMIN_USER=admin KEYCLOAK_ADMIN_PASSWORD=admin ./setup-seller-service-client.sh
```

09 Getting started

A first-run checklist, in order. Each step depends on the one before it.

1. **Copy env files.** Root `.env.example` → `.env` ; `frontend/.env.local.example` → `frontend/.env.local` . Fill in `GITHUB_TOKEN` if you need config-server to reach the external config repo (it's `optional:` , so this can wait).
2. **Bring up infra.** `docker compose up -d` from the repo root.
3. **Provision Keycloak** — run the three `keycloak/*.sh` scripts above, once, against the now-running Keycloak.
4. **Start config-server, then discovery-service.** Order matters — both from their own `services/<name>/` directory: `mvn spring-boot:run` .
5. **Start the remaining 8 services** (any order, but give each a moment to register with Eureka): customer, product, seller, order, payment, notification, cart, api-gateway.
6. **Start the frontend.** `cd frontend && npm install && npm run dev` , then visit `http://localhost:3000` .
7. **Walk the golden path:** sign in → browse → add to cart → checkout. Register as a seller, approve yourself as admin in Keycloak, list a product, buy it from a second account, fulfill the order, watch a payout appear.

WATCHING IT WORK

MailDev (`:10081`) shows every email the saga sends. Zipkin (`:9411`) traces a request across services. Grafana (`:4000`) has a pre-built dashboard once Prometheus has scraped at least one request through the saga.

10 Demo credentials & login

Every password below is a local-dev default, not a secret — defined as `${VAR:-default}` in `docker-compose.yml` and documented in root `.env.example`.

INFRASTRUCTURE / ADMIN TOOLS

TOOL	URL	USERNAME	PASSWORD
Keycloak admin console	localhost:8080	admin	admin
pgAdmin	localhost:8087	pgadmin@pgadmin.org	admin
Grafana	localhost:4000	admin	admin
mongo-express	localhost:8081	no login by default in this compose config	
Postgres (DB client)	localhost:5432	rajitha	rajitha
MongoDB (DB client)	localhost:27017	rajitha	rajitha
MinIO console	localhost:9001	rajitha	rajitha123

SIGNING INTO THE STOREFRONT (localhost:3000)

There's no pre-seeded demo account. The "Sign in" button goes through Keycloak's own login screen, which also has a **Register** link for creating a brand-new account on the spot. To get a working login, either:

1. Click **Sign in** → **Register** on the storefront and create a fresh account there, or
2. Create a user directly in the Keycloak admin console (`localhost:8080` → realm `micro` → Users → Add user, then set a password on the Credentials tab).

GETTING SELLER / ADMIN ACCESS

Any account created this way starts as a plain `customer`. To become a **seller**: sign in, hit "Sell with us", register, then have an admin approve it (`PATCH /api/v1/sellers/{id}/status` , or the `/admin/sellers` page once you have an admin account). To get an **admin** account, grant the `admin` realm role manually in Keycloak's admin console (Users → your user → Role mapping → Assign role) — nothing in the app can self-promote to admin.

11 Environment & secrets

Every credential across the stack is documented in root `.env.example`. `docker compose` auto-loads a root `.env` for the containers — but since the 10 services aren't containerized, running one via `mvn spring-boot:run` needs the relevant variables exported in that shell/IDE run config too. `.env` doesn't reach them automatically.

VARIABLE	USED BY	DEFAULT
POSTGRES_USER / PASSWORD	Postgres container	rajitha / rajitha
MONGO_ROOT_USER / PASSWORD	MongoDB + mongo-express	rajitha / rajitha
KEYCLOAK_ADMIN_USER / PASSWORD	Keycloak bootstrap admin, all setup scripts	admin / admin
GRAFANA_ADMIN_USER / PASSWORD	Grafana container	admin / admin
GITHUB_TOKEN	config-server → external config repo	empty
STRIPE_SECRET_KEY	payment-service, seller-service	empty (fails closed)
STRIPE_CONNECT_WEBHOOK_SECRET	seller-service webhook verification	empty
KEYCLOAK_ADMIN_CLIENT_SECRET	seller-service → Keycloak Admin API	empty
PLATFORM_COMMISSION_RATE	payment-service payout math	0.10
NEXT_PUBLIC_API_GATEWAY_URL	frontend → gateway	http://localhost:8222
NEXT_PUBLIC_SITE_URL	frontend SEO (sitemap, canonical, OG)	http://localhost:3000
NEXT_PUBLIC_GRAFANA_URL	admin dashboard's Grafana link	http://localhost:4000

Root `.gitignore` excludes `.env` / `.env.local` — only the `.example` files are ever committed.

12 Monitoring

`micrometer-registry-prometheus` on all 10 services, exposing `/actuator/prometheus`. Two hand-instrumented business counters sit at the saga's actual decision points — not just auto-instrumented HTTP/JVM metrics.

METRIC	WHERE	MEANING
<code>stock_reservation_total{result}</code>	product-service	Checkout reserves stock vs. loses to a stockout / optimistic-lock race
<code>payment_charge_total{result}</code>	payment-service	A reserved-stock order charges successfully vs. fails at Stripe

The pre-built Grafana dashboard (`monitoring/grafana/dashboards/ecommerce-overview.json`) also graphs gateway request rate and 5xx rate by route, JVM heap per service, Kafka consumer lag by group, HikariCP active connections, and service uptime.

NO ALERTING

Prometheus + Grafana are wired for dashboards and exploration, not paging — no Alertmanager, no alert rules configured.

13 Testing & CI

Three layers, deliberately not collapsed into one: fast Mockito unit tests everywhere, one Testcontainers-backed integration test for the saga's most critical wiring, and Playwright e2e specs for the browser-facing golden paths.

UNIT TESTS

Mapper tests + service-impl tests (Mockito/ `MockitoBean`) per service, plus one `*ApplicationTests` context-load test each.

INTEGRATION TEST

`order-service/ ... /saga/OrderSagaIntegrationTest.java` — real Postgres 16 + real Kafka (both Testcontainers), the actual `@KafkaListener` beans, real JSON (de)serialization, real DB transactions. Proves the wiring between consumers actually works, which a mocked unit test structurally can't. Scoped to order-service (the saga orchestrator) as the demonstrated pattern — the same `@DynamicPropertySource` + `@MockitoBean` approach transfers directly to product-service/payment-service if extended.

END-TO-END (PLAYWRIGHT)

SPEC	COVERS
<code>e2e/catalog.spec.ts</code>	Homepage loads or degrades gracefully, search narrows results, PDP opens
<code>e2e/auth-gates.spec.ts</code>	Every gated route actually redirects an anonymous visitor to Keycloak
<code>e2e/customer-checkout.spec.ts</code>	Sign in → add to cart → reach checkout (needs <code>E2E_CUSTOMER_*</code> credentials)
<code>e2e/seller-flow.spec.ts</code>	Active seller reaches overview + orders-to-fulfill (needs <code>E2E_SELLER_*</code> credentials)

CI PIPELINE

All 10 services' `Jenkinsfile` s run an explicit `mvn -B verify` stage before the shared-lib `buildService(...)` call, so a test failure fails the pipeline regardless of what that external step does internally. `frontend/Jenkinsfile` (new): install → lint → type-check → build → Playwright smoke.

14 Gotchas

Real bugs and environment traps found while building this — worth knowing before you hit them yourself.

Gateway's own actuator endpoint 401'd Prometheus's scrape

fixed

—

`SecurityConfiguration`'s `anyExchange().authenticated()` catch-all applies to api-gateway's own local endpoints, not just proxied traffic — `/actuator/prometheus` had no `permitAll()` matcher, so Prometheus's scrape request got a 401. Fixed with a `.pathMatchers("/actuator/**").permitAll()` matcher ahead of the catch-all.

`exchange.getPrincipal()` is empty inside a Gateway

GlobalFilter

—

Even on a fully-authenticated request, it reads empty. Use `ReactiveSecurityContextHolder.getContext().map(SecurityContext::getAuthentication)` instead — backed by the Reactor `Context` Spring Security actually writes into.

`RestClient` / `WebClient` URI templates percent-encode a full base URL

—

`.uri("{var}/path", value)` percent-encodes whatever `value` is — including `:/` and, for an IPv6 literal host, even `[ / ]`. Build the full URL as a string and pass `.uri(URI.create(fullUrl))` instead.

Keycloak's `admin-cli` client issues near-empty tokens

—

No `sub`, no `realm_access` — just `exp` / `iat` / `azp` / `scope`. Get a test token from a client with the default full scope set (like `nextjs-storefront`) instead, or you'll chase phantom "missing X-User-Id" bugs that are actually just a bad test token.

Gateway routing lives entirely outside this repo —

`spring.cloud.gateway.routes` is defined only in the external `e-com-app-config` repo — `services/api-gateway/src/main/resources/application.yml` defines zero routes. Adding a new service means adding its route there (or a local scratch override for testing).

product-service's Flyway migrations are stale – don't trust them —

`db/migration/V1 / V2` predate the `ProductVariant / sellerId /Category-CRUD/ ProductImage` schema entirely. Every schema change relies on Hibernate `ddl-auto: update` instead; local scratch configs explicitly set `spring.flyway.enabled: false`. Don't add new migrations without first reconciling the existing ones.

Windows Hyper-V reserves TCP port ranges Docker can't bind —

`netsh interface ipv4 show excludedportrange protocol=tcp` shows them — port `3001` fell inside one, so Grafana moved to `4000`. If a container's host port mysteriously refuses to bind with a permissions-flavored error, check this before assuming a Docker bug.

After a Docker Desktop crash, Kafka can fail to rejoin Zookeeper —

`org.apache.zookeeper.KeeperException$NodeExistsException` — a stale ephemeral broker-registration znode from before the crash. A plain restart isn't enough since it reuses the same container state; fix with `docker compose up -d --force-recreate zookeeper kafka`.

Playwright's browser binary downloads from a CDN, not npm —

`npm playwright install chromium` pulls from `cdn.playwright.dev` — if that's blocked or heavily rate-limited on a given network, the specs still type-check and lint clean, but `npm run test:e2e` can't actually run until the browser lands some other way.

Package naming:

exception

vs.

exeption

—

Every service uses `exception` except `customer-service` and `product-service`, which kept an original typo (`exeption`). Match the existing spelling within a given service rather than "fixing" it as a drive-by.

Only touch stock math in one place

—

Don't add stock-decrement/refund logic anywhere except `ProductServiceImpl.purchaseProductService` / `releaseStock` — the saga's stock-topic/stock-release-topic consumers call straight into these, so that's the one place "how do we mutate `ProductVariant.availableQuantity` " should live.

15 Build history

Sixteen phases, each shipped with its own tests and live verification. Order carries real information here — each phase built directly on the one before it.

0–5	Foundation Variant-aware catalog, order-state machine, Kafka saga replacing synchronous checkout, Redis cart, gateway hardening (CORS/rate-limit/circuit-breaker), the Next.js storefront itself.
6	Marketplace foundation seller-service, Keycloak realm roles, product ownership checks, category CRUD, gateway role-based routing.
7	Seller-aware saga & payouts <code>sellerId</code> threaded through the saga, <code>SellerPayout</code> ledger, Stripe Connect onboarding + settlement.
8	Order lifecycle Fulfillment states, refunds, customer-scoped order history, coupons.
9	Product images MinIO, presigned-upload flow, seller dashboard seed.
10	Reviews & ratings Verified-purchase gate across services, aggregate ratings, fail-closed on outage.
11	Seller dashboard Overview, orders-to-fulfill, shared status gate.
12	Admin dashboard Categories, seller moderation, all-orders view.
13	Observability Prometheus + Grafana + kafka-exporter, business metrics at saga decision points.
14	SEO Dynamic metadata, JSON-LD structured data, sitemap/robots.
15	Secrets hardening Root <code>.env.example</code> , parameterized docker-compose credentials, root <code>.gitignore</code> .
16	Testing & CI Testcontainers saga test, Playwright e2e, <code>mvn verify</code> in every Jenkinsfile, new frontend Jenkinsfile.

16 Locked decisions

Architectural calls made deliberately, not defaults — revisit only with a real reason.

- **Multi-vendor, not single-seller-admin.** Sellers are Keycloak users holding a role, not a separate identity system.
- **No order-splitting.** A checkout can mix products from multiple sellers; one `Order`, one Stripe charge, `OrderLine.sellerId` carries the split.
- **Payouts via Stripe Connect "separate charges and transfers."** One platform charge, then per-seller `Transfer` s off a flat commission rate — not real-time split payments.
- **Gateway is the only trust boundary.** Downstream services authorize off forwarded headers, never re-validate a JWT.
- **MinIO, not a live cloud bucket.** Self-hosted, local — matches the local/dev-environment scope.
- **Docker Compose / single VM as the hosting target** — observability and secrets tooling is scoped to that, not Kubernetes.
- **Flat `Category`, no hierarchy.** Revisit only if the catalog actually grows multi-level.

Cloth Shop Platform — internal engineering reference, generated from the codebase's own `CLAUDE.md` and `PLAN.md`. Reflects the state after Phase 16.